

TRPO 实验报告

南京大学

施静宇 2026 年 8 月 6 日

1 引言

Trust Region Policy Optimization (TRPO) 通过限制相邻两次策略之间的差异，缓解策略梯度方法因更新幅度过大而产生的训练不稳定问题。本次任务基于给定代码框架，补全 trajectory 数据收集、rollout 与 trajectory 两种模式下的广义优势估计 (Generalized Advantage Estimation, GAE)，以及包含共轭梯度和回溯线搜索的策略更新过程。实验在 HalfCheetah-v5、Hopper-v5 和 Walker2d-v5 三个 MuJoCo 连续控制环境上进行，并以 HalfCheetah-v5 为基准，分别比较 Advantage Normalization 的开关以及 trajectory、rollout 两种采样方式。

2 代码补全与理论问题

本项目的主要入口为 `run_trpo.py`。与本次任务直接相关的代码位于 `algorithm/baseonpolicy.py`、`algorithm/trpo.py` 和 `memory/memory.py`：其中 `collect_trajectories()` 负责按完整轨迹收集数据，`compute_advantages_from_traj()` 与 `compute_advantages_from_rollout()` 分别计算两种数据组织形式下的 advantage 和 return，`_update_policy()` 完成 actor 与 critic 的更新。以下问题按照任务说明中的顺序排列。

2.1 `collect_trajectories` 实现部分

1. `collect_trajectories` 与普通 rollout 采样方式相比，数据收集的停止条件分别是什么？

答：`collect_trajectories` 停止条件是收集完完整的 `trajnum` 个 trajectories。

2. 在 trajectory 采样模式下，每一条轨迹需要保存哪些信息？这些信息分别在后续 TRPO 更新中起什么作用？当环境返回终止信号时，轨迹又会如何结束？未达到最大长度的部分又应如何处理？

答：每一条轨迹需要收集 `states, actions, dones, log_probs`，在后续 TRPO 更新过程中 `states, actions, dones` 用来计算 advantage，而 `log_probs` 在后续 importance sampling 中用来计算 ratio probability。

当环境返回终止信号时，程序跳出 `while not self.traj_rollout.full`：循环，停止记录轨迹信息，未达到最大长度的部分通过 padding 补零处理。

3. `TrajectoryRollout` 中的 `masks` 变量有什么作用？如果不使用 `masks`，计算 return 或 advantage 时可能出现什么问题？

答：`masks` 作用是通过 0 和 1 的标注可以判断出真实轨迹长度，后续更是通过与 `values` 做乘法，防止 padding 位置通过 critic 神经网络后变成非零值。否则 value return advantage 在计算时都有可能在 padding 位置出现非零值，造成误差。

4. `collect_trajectories` 在收集完一批轨迹后为什么需要重新初始化环境状态？

答：主要是因为本代码实现时，是通过多个并行环境收集 trajectory 数据的，可能的情况是某个环境恰好收集完 `trajnum` 条数据，其他环境恰好收集到一半，此时如果不初始化环境状态，下一批数据收集时就可能从 episode 的一半开始，造成误差。

2.2 _update_policy 实现部分

TRPO 的理论分析依赖于策略性能改进下界。令

$$\alpha = D_{\text{TV}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}}), \quad \epsilon = \max_{s,a} |A^{\pi_{\text{old}}}(s,a)|,$$

则 TRPO 论文 Theorem 1 给出

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{4\epsilon\gamma}{(1-\gamma)^2} \alpha^2. \quad (1)$$

1. 解释式 (1) 中每一项的实际意义。该公式在什么情况下可以保证策略的单调改进？由此直观解释 TRPO 算法的设计思路，包括优化目标和约束的设计。

答： $\eta(\pi_{\text{new}})$ 是新策略的期望折扣回报； $L_{\pi_{\text{old}}}(\pi_{\text{new}})$ 指的是将新策略状态分布看成旧策略时对 $\eta(\pi_{\text{new}})$ 的近似，叫 surrogate objective；该公式优化的总体思路是希望式 (1) 的下界能够大于等于 $\eta(\pi_{\text{old}})$ ；

由于 $L_{\pi_{\text{old}}}(\pi_{\text{old}}) = \eta(\pi_{\text{old}})$ ，所以，在保证：surrogate objective 有所提升；新旧策略之间的距离不会过大；surrogate objective 的提升足以覆盖近似新旧状态分布引起的误差时；可以使得新策略下界大于等于旧策略。

理论算法由此通过最大化含有最大 KL 散度惩罚的性能下界来获得严格的单调改进保证。实际 TRPO 则以提升 surrogate objective 为目标，并使用平均 KL trust region 限制新旧策略之间的变化，从而近似实现这一理论思想并提高策略更新的稳定性，这样在工程上可实现性更强，但由于有限样本估计和局部优化近似，不再具有完全严格的单调改进保证。

2. 和文章 *Approximately Optimal Approximate Reinforcement Learning* 中的 Theorem 4.1 (即 TRPO 论文中的公式 (6)) 作对比，TRPO 作者在分析上的处理技术有什么具体不同，使得 TRPO 可以推广到所有一般的随机策略类，而不是给定的混合策略类？

答：在 Kakade & Langford 论文中， α 是显示混合系数，其证明依赖于以 $1 - \alpha$ 的概率使用旧策略，以 α 概率使用待选策略这一特殊结构；而 Theorem 1 中混合系数 α 被定义为新旧策略的最大总变差散度，利用总变差散度耦合的性质使得新旧策略可以在联合采样，使得任意状态下，新旧策略都以至少 $1 - \alpha$ 的概率选择相同的动作，进而可以耦合两条轨迹，并用“此前是否已经出现策略分歧”来控制新旧策略状态分布之间的差异。

相较于原结论，TRPO 将“显式混合系数”替换为“任意两个策略之间的最大 TV 距离”，同时相应地放宽 advantage 上界并调整常数。因此，新策略不必写成旧策略与某个候选策略的线性混合，只要它与旧策略足够接近，理论下界就适用，这使得 TRPO 算法能够覆盖神经网络参数化的高斯策略等一般随机策略类。

3. TRPO 中策略更新的 surrogate objective $L_{\pi_{\text{old}}}(\pi_{\text{new}})$ 一项在实际算法中是如何设计的？

答：TRPO 论文中的 $L_{\theta_{\text{old}}}(\theta) = \eta(\theta_{\text{old}}) + \sum_s \rho_{\theta_{\text{old}}}(s) \sum_a \pi_{\theta}(a|s) A_{\theta_{\text{old}}}(s,a)$ 涉及对全部状态、全部动作的求和，实践中要靠采样估计。论文做了三处替换：

1. 把 $\sum_s \rho_{\theta_{\text{old}}}(s)[\cdot]$ 换成期望 $\frac{1}{1-\gamma} \mathbb{E}_{s \sim \rho_{\theta_{\text{old}}}}[\cdot]$ ；
2. 把优势 $A_{\theta_{\text{old}}}$ 换成 $Q_{\theta_{\text{old}}}$ 两者只差一个不依赖 θ 的常数 $V_{\theta_{\text{old}}}(s)$ ，同样不影响 $\arg \max$ ，但用 Q 更方便直接从回报进行估计；
3. 把对动作的求和 $\sum_a \pi_{\theta}(a|s) Q_{\theta_{\text{old}}}(s,a)$ 换成重要性采样估计 $\mathbb{E}_{a \sim q} \left[\frac{\pi_{\theta}(a|s)}{q(a|s)} Q_{\theta_{\text{old}}}(s,a) \right]$ ，其中 q 是实际采样动作作用的分布。

最终得到可以直接用蒙特卡洛样本估计的优化问题：

$$\underset{\theta}{\text{maximize}} \quad \mathbb{E}_{s \sim \rho_{\theta_{\text{old}}}, a \sim q} \left[\frac{\pi_{\theta}(a|s)}{q(a|s)} Q_{\theta_{\text{old}}}(s, a) \right] \quad \text{s.t.} \quad \mathbb{E}_{s \sim \rho_{\theta_{\text{old}}}} [D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s) \parallel \pi_{\theta}(\cdot|s))] \leq \delta.$$

在最终的代码中 surrogate objective 的等价实现是： $\hat{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)} \hat{A}_t$ 。

4. 为什么 TRPO 需要约束新旧策略之间的 KL divergence，而不是定理中的 TV 距离？实际算法采用 KL 散度的二阶近似

$$D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_{\theta}) \approx \frac{1}{2} \Delta \theta^{\top} (\nabla_{\theta}^2 D_{\text{KL}}) \Delta \theta.$$

延续该思路，解释为什么 Algorithm 1 的第 8、9 行还需要使用 line search 搜索参数。

答：TV 距离含绝对值，不够光滑，难以计算梯度，所以很难直接在神经网络中优化，同时很难在连续动作空间计算；而 KL divergence 关于常用的高斯策略参数可微，同时由 Pinsker 不等式 $D_{\text{TV}}(p, q)^2 \leq \frac{1}{2} D_{\text{KL}}(p \parallel q)$ ，控制 KL divergence 的同时也可以间接控制 TV 距离，因此实际算法使用 KL 作为 trust region 约束。

结合 surrogate objective 的一阶近似，可以求得满足近似 KL 约束的自然梯度方向和 full step。然而，该结果只保证满足局部近似问题；由于有限样本估计误差、KL 二阶近似以及共轭梯度的产生的误差，直接采用 full step 后，真实 KL 仍可能超过阈值，真实 surrogate objective 也可能没有提高。因此 Algorithm 1 第 8、9 行使用回溯线搜索，沿 full step 逐步缩小更新幅度，并使用未近似的经验 KL 和 surrogate objective 检查候选参数。

2.3 Advantage 与 Return 的计算

两种采样模式均使用 GAE。对第 t 个 transition，TD residual、advantage 和 return 的基本形式为

$$\delta_t = r_t + \gamma(1 - d_t)V_{\phi}(s_{t+1}) - V_{\phi}(s_t), \quad (2)$$

$$\hat{A}_t = \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1}, \quad (3)$$

$$\hat{R}_t = \hat{A}_t + V_{\phi}(s_t). \quad (4)$$

Rollout 模式。 Rollout 数据按固定步数收集。设并行环境数为 N ，每个环境采集 T 步，则 `states` 的形状为

$$(N, T, \text{state_dim}),$$

而 `rewards`、`done`s 和计算得到的 `advantages`、`returns` 的形状均为 (N, T) 。代码首先将所有状态展平并批量输入 critic，得到各时刻的价值估计 $V_{\phi}(s_t)$ 。由于固定步数采样可能在 episode 尚未结束时停止，还需对采样结束后的最新状态 `self.observations` 计算

$$V_{\phi}(s_T),$$

作为反向递推开始时的 `last_values`，从而对未结束的 rollout 末端进行 bootstrap。

随后代码从 $t = T - 1$ 反向递推。若第 t 步未终止，则 $d_t = 0$ ，TD residual 会使用下一状态价值；若该步为 episode 终止位置，则 $d_t = 1$ ，因子 $1 - d_t$ 将 bootstrap 和后续 advantage 递推同时截断，避免不同 episode 之间的信息相互传播。最后以

$$\hat{R}_t = \hat{A}_t + V_{\phi}(s_t)$$

构造 critic 的回归目标。

Trajectory 模式。 Trajectory 模式的数据形状为

$$(\text{trajnum}, \text{max_episode_length}, \cdot).$$

由于不同 trajectory 的实际长度不同，数组中较短轨迹之后的位置为 padding，并由 **masks** 标记：真实 transition 对应 $m_{i,t} = 1$ ，padding 对应 $m_{i,t} = 0$ 。代码先批量计算所有状态的价值，再令

$$V_{i,t} \leftarrow V_{i,t} m_{i,t},$$

防止 padding 位置经过 critic 后产生非零价值。

对于第 i 条轨迹，其真实长度由

$$T_i = \sum_t m_{i,t}$$

得到。代码只在 $t = T_i - 1, \dots, 0$ 的有效范围内反向计算 GAE，padding 位置的 advantage 和 return 始终保持为零。每条已保存 trajectory 的末端初始化为 **next_values=0** 和 **last_advantages=0**，因此当前实现将轨迹末端视为终止边界，不再使用末端之后的价值进行 bootstrap。计算完成后，在策略更新阶段还会再次利用 **masks** 删除所有 padding，只保留真实 transition。

两种模式的主要差异。 两种模式使用相同的 GAE 递推公式，主要区别在于数据边界和样本组织方式。Rollout 每次使用固定数量

$$N_{\text{rollout}} = N \times T$$

的 transition，batch 大小固定；其采样末端可能位于 episode 中间，因此需要利用最新状态价值进行 bootstrap。Trajectory 每次固定收集若干条结束的轨迹，有效 transition 数

$$N_{\text{traj}} = \sum_i T_i$$

随各轨迹长度变化；每条轨迹分别反向递推，并使用 **masks** 排除 padding，其末端在当前实现中取下一状态价值为零。因此，rollout 的主要特点是样本数固定且末端依赖 critic 的价值估计，而 trajectory 的主要特点是轨迹边界清晰，但每次更新的有效样本数不固定，并需要额外处理 padding。

3 实验设置

3.1 运行环境与公共超参数

正式实验在矩池云平台完成，使用 NVIDIA RTX 3090 24 GB GPU 和 12 核 CPU。软件环境为 Python 3.11、PyTorch 2.6.0 (CUDA 12.4)，训练环境采用 Gymnasium MuJoCo v5。TRPO 的网络规模较小，主要开销来自 MuJoCo 环境交互和并行环境管理，因此 GPU 利用率不是衡量训练是否正常的主要依据。所有正式实验均采用随机种子 0，并使用 SwanLab 记录训练过程。

表 1 正式实验的公共超参数

超参数	取值	超参数	取值
总训练 epoch 数	5000	并行训练环境数	8
最大 episode 长度	1000	测试 episode 数	10
折扣因子 γ	0.99	GAE 参数 λ	0.95
KL 约束阈值 δ	0.01	共轭梯度迭代次数	10
回溯线搜索次数	10	回溯系数	0.5
Actor/Critic 隐层	[64, 64]	隐层激活函数	Tanh
Critic 学习率	10^{-3}	Critic 优化器	Adam
Critic 更新轮数	5	Critic batch size	256
测试间隔	50 epoch	训练日志间隔	10 epoch
模型保存间隔	1000 epoch	动作采样	随机策略采样

Actor 输出高斯分布的均值和可学习标准差，并通过 Tanh 变换及仿射变换映射到环境动作范围；Critic 输出标量状态价值。除专门的消融实验外，默认开启 Advantage Normalization。

3.2 实验配置与比较口径

本报告共使用五组正式实验。五组实验共享同一个 HalfCheetah rollout baseline，从而在控制计算量的同时分别完成环境比较、采样方式比较和 Advantage Normalization 消融。具体配置见表 2。

表 2 五组正式实验配置

编号	环境	采样方式	advan_norm	主要用途
E1	HalfCheetah-v5	rollout	true	公共 baseline；参与三环境比较、采样方式比较和归一化消融
E2	Hopper-v5	rollout	true	与 E1、E3 比较不同环境的训练表现
E3	Walker2d-v5	rollout	true	与 E1、E2 比较不同环境的训练表现
E4	HalfCheetah-v5	trajectory	true	与 E1 比较 trajectory 和 rollout
E5	HalfCheetah-v5	rollout	false	与 E1 比较 Advantage Normalization 的影响

rollout 模式设置 `interact_per_epoch=2000`，8 个并行环境每次策略更新固定收集

$$N_{\text{rollout}} = 8 \times 2000 = 16000$$

个 transition。trajectory 模式每次收集 `trajnum=10` 条完整轨迹，有效 transition 数为

$$N_{\text{trajectory}} = \sum_{i=1}^{10} T_i,$$

其中 T_i 为第 i 条轨迹的真实长度，因此每次更新的样本量并不固定。比较两种采样方式时应优先以 `interaction_step` 为横轴，而不能只按 epoch 对齐。

本文主要使用 evaluation episode reward 衡量策略性能，并结合训练阶段的 reward、KL divergence、surrogate objective、backtracking step 和 critic loss 分析优化过程。由于三种环境的 reward 定义与数值尺度不同，环境难度不能仅由最终 reward 的绝对大小判断，还需结合学习速度、波动程度和训练过程是否形成稳定平台。所有实验均只运行一个随机种子，因此后文结论属于当前实现和配置下的描述性观察，不作统计显著性声明。

4 实验结果与分析

4.1 不同环境的训练结果

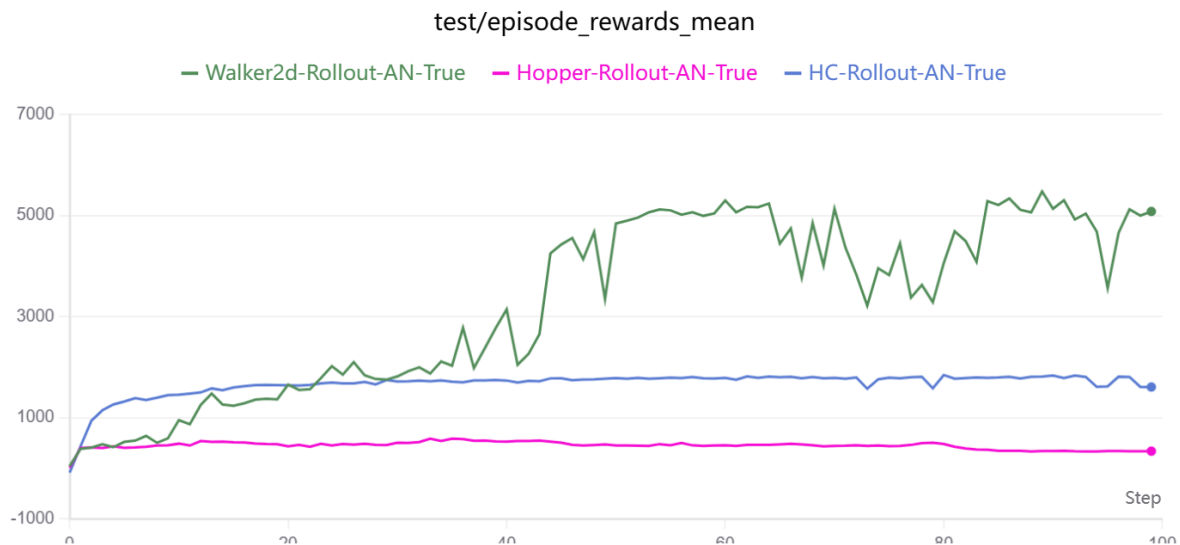


图 1 TRPO 在 HalfCheetah-v5、Hopper-v5 和 Walker2d-v5 三个环境上的 evaluation mean reward 曲线

在相同的 rollout 采样方式和 `advan_norm=True` 的设置下，三个环境的测试平均回报呈现出明显不同的变化趋势。HalfCheetah-v5 的回报在训练初期迅速上升，由接近 0 提升至约 1200，随后继续缓慢增长，并在约 1600–1800 区间内保持相对稳定，整体收敛速度较快、波动较小。Walker2d-v5 前期提升较慢，但在中期出现明显跃升，最高达到约 5000；不过其后期波动较大，多次从 5000 附近下降至 3000–4000 区间，说明虽然获得了较高回报，但训练稳定性相对较弱。Hopper-v5 的回报在初期升至约 400–500 后基本进入平台，后期还出现一定下降，最终约为 300，说明当前统一超参数下其后续性能提升较为有限

4.2 Advantage Normalization 的影响

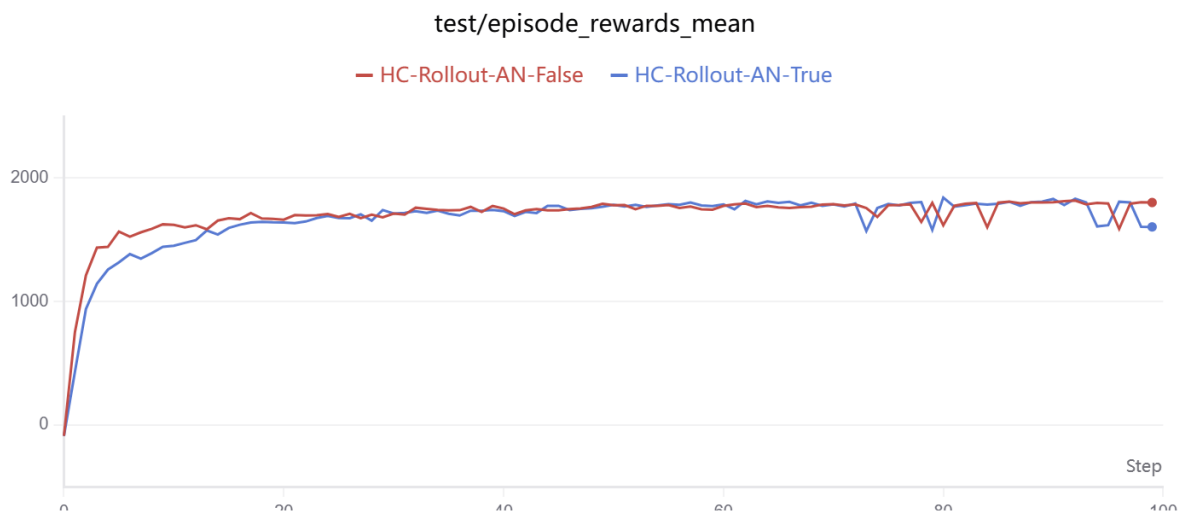


图 2 HalfCheetah-v5 中 Advantage Normalization 的回报对比

如图 2 所示，在 HalfCheetah-v5 的 rollout 采样模式下，开启与关闭 advantage normalization

的测试平均回报整体差异较小。未归一化组在训练初期提升略快，较早达到约 1500 的回报；归一化组随后逐渐追上，两组中后期均稳定在约 1700 1800 区间。训练后期两组都出现少量短暂回落，其中归一化组末端下降更明显，但从整体趋势看，advantage normalization 并未带来显著的最终性能提升。

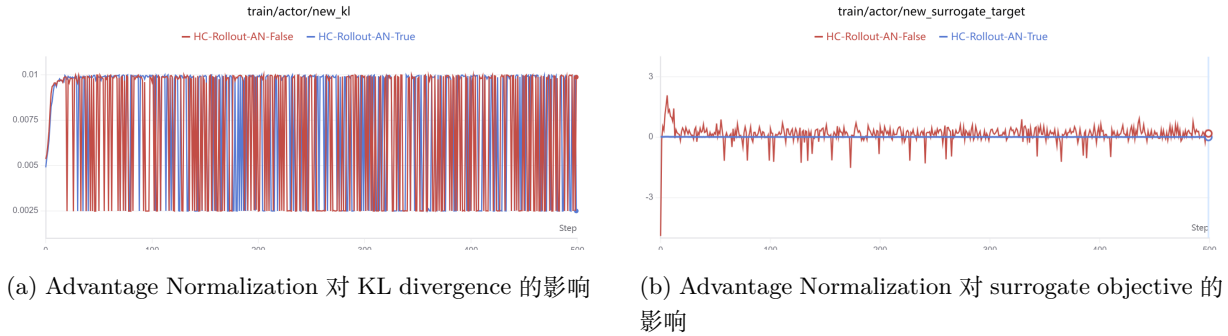


图 3 HalfCheetah-v5 中 Advantage Normalization 对 TRPO 策略更新的影响

图 3(a) 表明，两组实验的实际 KL divergence 大部分时间都接近设定的 trust-region 阈值 $=0.01$ ，并间歇性下降到约 0.0025。两组曲线高度重合，说明无论是否进行 advantage normalization，KL 约束与回溯线搜索都能有效限制策略更新幅度，归一化对实际 KL 的直接影响较小。

图 3(b) 中，归一化组的 surrogate objective 基本维持在零附近，波动幅度较小；未归一化组则表现出更明显的正负波动，并在训练初期及部分更新中出现较大的尖峰。这说明 advantage normalization 能够统一 advantage 的数值尺度，使 surrogate objective 的数值表现更加稳定。不过，这种数值稳定性并未在本实验中转化为明显更高的测试回报。因此，在 HalfCheetah-v5 上，advantage normalization 的主要作用更接近于改善优化过程的数值稳定性，而不是必然提高最终策略性能。

4.3 Trajectory 与 Rollout 的比较

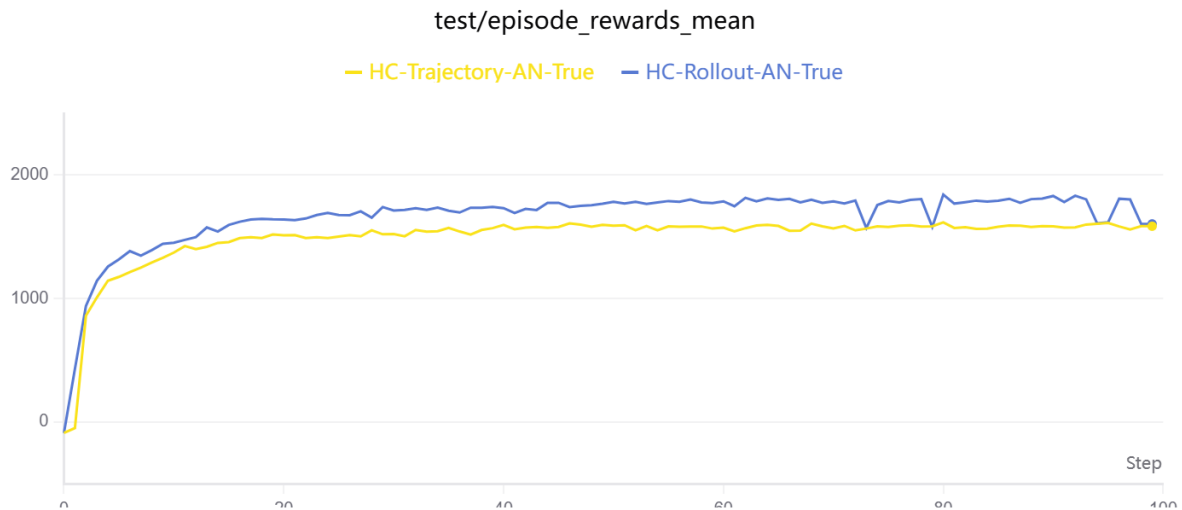


图 4 HalfCheetah-v5 中 trajectory 与 rollout 的回报对比

如图 4 所示，在 HalfCheetah-v5 环境和 `advan_norm=True` 设置下，rollout 与 trajectory 两种采样方式都能在训练初期迅速提高测试平均回报。rollout 组整体上升更快，并在中后期大多保持在约 1700 1800 的区间；trajectory 组则较早进入约 1500 1600 的平台，后续提升幅度有限。

因此，从该图的大部分训练阶段来看，rollout 获得了更高的测试回报

表 3 HalfCheetah-v5 中两种采样方式的 transition 使用情况

采样方式	实际交互数	有效 transition 数	样本利用率
Rollout	16000	16000	100%
Trajectory	16000	10000	62.5%

5 实验探索问题

5.1 综合探索问题

1. 在相同超参数设置下，三个环境的训练难度是否一致？请结合 reward 曲线进行分析。

答：相同超参数设置下，三个环境训练难度并不一致，具体趋势分析见图 1 注释。

总的来说,HalfCheetach reward 曲线最稳定,进入平台期后波动最小说明该参数下 HalfCheetach 训练难度最低。

Walker reward 曲线波动最大，多次出现大幅度下降，说明难以进行稳定训练，该超参数设置下，该环境训练难度最大。

2. 在 trajectory 模式和 rollout 模式下，advantage 与 return 的计算方式有什么不同？

答：在 trajectory 模式下由于 padding 补齐机制，计算 advantage 和 return 前需要判断 trajectory 真实 step 数量，经过 critic 网络计算 value 后还要把 padding 位置变为 0，随后分别对每条轨迹有效部分反向计算 advantage。

在 rollout 模式下，每个并行环境都固定收集相同数量的 transition，因此不存在 padding，所有位置均为有效样本。由于 rollout 可能在 episode 尚未结束时达到固定采样步数，计算最后一步的 TD residual 时，需要使用采样结束后的状态价值 $V_\phi(s_T)$ 进行 bootstrap；对于 rollout 中间出现的 episode 终止位置，则通过 $(1 - d_t)$ 截断 advantage 的递推。

5.2 Advantage Normalization 的影响

1. 请分别设置 `advan_norm=true` 和 `advan_norm=false` 进行实验。两组实验的平均回报曲线有何差异？

答：如图 2 所示，在 HalfCheetah-v5 的 rollout 采样模式下，开启与关闭 advantage normalization 的测试平均回报整体差异较小。未归一化组在训练初期提升略快，较早达到约 1500 的回报；归一化组随后逐渐追上，两组中后期均稳定在约 1700 1800 区间。训练后期两组都出现少量短暂回落，其中归一化组末端下降更明显，但从整体趋势看，advantage normalization 并未带来显著的最终性能提升。

2. 加入 advantage normalization 后，训练过程是否更加稳定？请结合 reward 曲线、KL divergence 或 surrogate objective 的变化进行分析。

答：reward 与 KL 曲线中，两组实验的差异均不明显，两组 reward 相近，说明归一化没有带来明显的最终性能的差异，KL 曲线相近说明 TRPO 的 trust-region 步长缩放和 line search 抵消了 advantage 变化，使两组更新幅度都被限制在相近范围内。

关闭归一化后，advantage 保持着原尺度，数值可能较大，不同 batch 间变化也较大，导致 surrogate objective 曲线出现了较大波动和尖峰。开启归一化后 advantage 被调整为均值接近 0、标准差接近 1，surrogate 的数值自然会更集中。

从 reward 曲线来看，是否归一化对训练稳定性影响不是很大。

3. 在你选择的测试环境中，advantage normalization 是否总是带来性能提升？如果不是，可能的原因是什么？

答：在本实验选择的 HalfCheetah-v5 环境中，advantage normalization 并未始终带来性能提升。开启和关闭归一化后，两组测试平均 reward 曲线整体接近，中后期均稳定在相似区间。

可能的原因是：TRPO 本身已经通过 KL 约束、自然梯度步长缩放和 line search 控制 Actor 的更新幅度，因此 advantage 数值的变化会在一定程度上被后续的步长调整抵消，即 advantage 放大后，自然梯度方向向量也同比放大，但为了满足相同的 KL 阈值 δ ，TRPO 会相应减小缩放系数，在理想条件下正好抵消。

5.3 Trajectory 与 Rollout 的样本利用率比较

1. 在 rollout 模式下，每次策略更新实际使用了多少个 transition？在 trajectory 模式下，每次策略更新又实际使用了多少个有效 transition？

答：前者使用了 16000 个，后者也使用了 16000 个，但实际有效的只有 10000 个。

2. 对比 trajectory 与 rollout 两种模式，哪一种样本利用率更高？

答：显然是 rollout 模式样本利用率更高。

原因在于 trajectory 模式判断轨迹收集结束的标准是收集的 trajectory 数量达到 `trajnum`，由于并行环境的存在，此时记录在其他环境中的 transition 数据为了不影响下一轮 episode 的收集要被初始化，这样就造成了样本利用率的下降，同时造成了一定的计算资源的浪费。

3. 样本利用率更高是否一定意味着训练效果更好？请结合实验曲线进行分析。

答：rollout 模式样本利用率较高，相同时间策略获得的信息更多，整体学的更快，所以最终 reward 更高。但由于 rollout 模式 trajectory 是人工截断，计算最后时刻状态的 value 时需要 bootstrap，产生了 bias。而 trajectory 模式样本利用率低，所以 reward 较低，但是保存了完整的 trajectory 信息，bias 较小，所以平台期波动较小，整体训练的更稳定。

6 总结与反思

1. 在本项目中，`collect_trajectories` 的实现是否会改变 TRPO 的 on-policy 特性？说出你的判断理由。

答：`collect_trajectories` 的实现不会改变 TRPO 的 on-policy 特性。在一次数据收集过程中，Actor 参数保持固定，所有并行环境均使用同一个当前策略生成动作，直到收集到规定数量的完整轨迹后才进行策略更新。因此，本批样本均来自更新前的策略 $\pi_{\theta_{old}}$ 。

为了维护 on-policy 假设，我们也必须在 trajectory 收集完之后重置环境，否则并行环境的存在可能导致某次 episode 先用旧策略收集再用新策略收集，导致不同策略的数据混在一个 batch 中。

2. 如果只能保留一种采样方式，你会选择 trajectory 还是 rollout？请结合样本利用率和训练性能说明理由。

答：很难在这两中采样方式中分出绝对的优劣，两者均保持了 TRPO 的 on-policy 特性，各有优缺点，从本实验看来，rollout 模式样本利用率更高而 trajectory 模式训练的稳定性更强。

如果计算资源有限而数据量庞大，环境交互成本较高，需要充分利用并行环境，或希望每次更新的数据规模固定时可以选择 rollout 模式。

如果要求训练过程稳定性高，重视完整轨迹统计、终止边界分析，可以选择 trajectory 模式。

3. 完成实验报告的撰写后，你对 TRPO 算法是否有了新的感悟或认识？

答：TRPO 论文描述了策略优化的长期痛点，即实践中难以控制梯度更新的步长，策略更新的稍大就会造成断崖式下跌，所以该论文的主线就是寻找一种能稳定训练的，同时有近似单调性能改进的策略更新方法。

在我看来 TRPO 算法先通过理论论证单调策略改进定理 (Theorem 1) 再为了工程可实现性进而做出了许多近似与改进的方案体现了一种算法上螺旋递进的美感。TRPO 并不是直接保证每次更新后的实际回报严格上升，而是通过 surrogate objective, KL trust region, 共轭梯度求解自然梯度方向和回溯线搜索，将一次策略更新控制在局部近似较为可靠的范围内。因此，TRPO 的稳定性来自一整套相互配合的机制，而不是单独依赖某一个 KL 阈值。

同时，实验也使我认识到，算法的实际表现不仅由理论决定，还会受到采样方式，advantage 估计（是否归一化）和有限样本误差等实现细节影响。比如，rollout 与 trajectory 模式在样本利用率，reward 曲线方面表现不同；advantage normalization 能改善 advantage 的数值尺度，却不一定直接提高最终回报。

总而言之，TRPO 的主要价值并不只是获得更高的回报，而是为策略优化提供了一种在受控范围内改进策略的方法。当然了，为了让步于工程的可实现性，算法进行了许多近似处理，使得本算法计算开销比较大，同时代码实现也比较复杂。

4. 针对本次算法实现，你是否有什么想法和改进建议？实现过程中是否因项目 README.md 或实验报告引导与介绍问题卡住较长时间？如果有，请详细说明并给出建议。你认为代码和实验报告问题的难度如何？

答：该项目的实现我觉得难点并不在算法的理解上，TRPO 论文写得层层递进，可以充分理解每个行为的动机，进而深入理解算法，同时数学证明也不是很难理解。

我觉得耗时最多的地方是在代码实现，相比于 DQN 算法，本项目在代码上更复杂。可能我目前接触体量如此大的代码项目的经历太少了，所以在代码上觉得困难，需要反复查看 utils.py, memory.py, baseonpolicy.py 等文件才能找到需要使用的函数，同时也借助了 AI 工具才得以完成。

7 总体结论

1. 在统一超参数下，HalfCheetah-v5 的学习速度较快且训练过程相对稳定；Hopper-v5 较早进入低回报平台；Walker2d-v5 能达到较高回报，但中后期波动最为明显。由于三个环境的奖励函数和数值尺度不同，不能仅根据最终回报的绝对大小判断训练难度。
2. 在 HalfCheetah-v5 中，Advantage Normalization 未带来明显的最终性能提升，两组 reward 与 KL 曲线整体接近。归一化主要使 surrogate objective 的数值尺度更加规整，说明其改善了优化过程的数值条件，但这种改善不一定会直接转化为更高的测试回报。

3. Rollout 模式采用固定长度采样，样本数量稳定且利用率较高，本实验中取得了更高的测试回报；trajectory 模式能够保留完整轨迹边界，回报曲线波动较小，但会因并行环境中的未完成数据被丢弃而降低样本利用率。因此，两种方式体现了样本利用效率与完整轨迹处理之间的权衡。
4. TRPO 的实际稳定性并非仅来自 KL 约束，而是由 surrogate objective、自然梯度、共轭梯度求解和回溯线搜索共同实现。与此同时，有限样本、critic 估计误差以及采样边界处理都会使理论保证在代码中只能近似成立，说明算法表现同时受到理论设计和工程实现细节的影响。

参考文献

- [1] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, “Trust Region Policy Optimization,” *Proceedings of the 32nd International Conference on Machine Learning*, pp. 1889–1897, 2015.
- [2] S. Kakade and J. Langford, “Approximately Optimal Approximate Reinforcement Learning,” *Proceedings of the 19th International Conference on Machine Learning*, pp. 267–274, 2002.
- [3] OpenAI, “Spinning Up: Trust Region Policy Optimization.” <https://spinningup.openai.com/en/latest/algorithms/trpo.html>.
- [4] Farama Foundation, “Gymnasium MuJoCo Environments.” <https://gymnasium.farama.org/environments/mujoco/>.
- [5] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Madry, “Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO,” *International Conference on Learning Representations*, 2020.